

Parameter Server vs. All-Reduce en entrenamiento distribuido de redes neuronales: diseño y evaluación experimental en O.N.O.

Lorenzo Minervino
Facultad de Ingeniería, Universidad de Buenos Aires
Aprendizaje Profundo
Junio 2026

Resumen—El entrenamiento distribuido de redes neuronales se apoya en dos familias de algoritmos para agregar gradientes: el *Parameter Server* (PS), con servidores centrales que concentran y actualizan los pesos, y *All-Reduce* (AR), donde los workers reducen gradientes entre sí sin servidor central. Se presenta O.N.O., un sistema de entrenamiento distribuido propio que implementa ambas estrategias. Sobre O.N.O. se comparan AR y PS (este último sincrónico, con barrera y con los pesos shardeados entre dos servidores) en MNIST y FashionMNIST, bajo un criterio de comparación justa a *igual número de workers*, midiendo convergencia, throughput y escalabilidad. La evidencia observada indica que, en este entorno homogéneo, sincrónico y sobre una sola máquina, All-Reduce y Parameter Server *convergen a la misma accuracy* (resultado esperable, ya que el régimen sincrónico hace que ambos apliquen la misma regla de actualización), mientras que All-Reduce alcanza esa accuracy en menos tiempo, debido a que el servidor de PS introduce un cuello de serialización. Al sumar nodos, PS amortiza su costo fijo y su throughput crece con pendiente algo mayor; sus ventajas estructurales (sharding de modelos que no entran en un nodo, heterogeneidad, asincronía) no se ejercitan en este entorno. Las conclusiones se acotan a un clúster simulado sobre una única máquina de ocho núcleos con red rápida.

I. INTRODUCCIÓN

A medida que crecen los modelos y los datos, el entrenamiento de redes neuronales se vuelve inviable en un único proceso y se distribuye sobre varios *workers* (los procesos que calculan gradientes sobre una partición local de los datos) que trabajan en paralelo (SGD distribuido con paralelismo de datos) [1]. El cuello de botella deja de ser el cómputo y pasa a ser la *comunicación*: cómo se agregan los gradientes y cómo se sincronizan los pesos. Dos enfoques dominan. El *Parameter Server* mantiene los pesos en uno o más servidores; los workers envían gradientes y reciben parámetros actualizados [2]. *All-Reduce* elimina el servidor: los workers intercambian y promedian gradientes entre sí, típicamente en anillo [3], [4], y cada uno aplica el mismo update.

Pregunta de investigación. ¿Cuándo conviene cada enfoque, por qué, y qué compromisos aparecen entre convergencia, throughput, escalabilidad y sincronización? No se busca el estado del arte en visión, sino caracterizar el *sistema* de entrenamiento.

Contribución. (i) Se describe O.N.O., un sistema distribuido propio que implementa tanto All-Reduce como Parameter Server; (ii) se aporta una comparación experimental honesta

AR vs. PS sobre MNIST y FashionMNIST con un criterio de comparación justa (igual número de workers); y (iii) se propone una matriz de decisión que indica cuándo preferir cada enfoque, acotada a las condiciones evaluadas.

II. FUNDAMENTOS Y DISEÑO

II-A. SGD distribuido con paralelismo de datos

Con W workers, cada uno procesa un mini-batch local de tamaño b (el `batch_size` por worker) y calcula un gradiente g_w . El gradiente agregado es el promedio $\bar{g} = \frac{1}{W} \sum_w g_w$, que equivale a un paso de SGD con *batch efectivo* $W \cdot b$: el tamaño de batch que “ve” realmente el optimizador tras agregar a los W workers. Promediar (y no sumar) mantiene la magnitud del gradiente (y por consiguiente el learning rate efectivo) independiente de W . Esta distinción es central: en O.N.O. el `batch_size` es *por worker*, de modo que, con b constante, agregar workers cambia el batch efectivo (Sección III-C).

II-B. Parameter Server

Los servidores almacenan los pesos, shardeados entre ellos cuando hay más de uno; los workers hacen *push* de gradientes y *pull* de parámetros [2]. El régimen de sincronización define el comportamiento: con *barrera* todos los workers esperan que el servidor aplique el update (sincrónico, equivalente estadístico a AR); sin barrera, los workers proceden con parámetros potencialmente sobre-especializados sobre su propia partición del dataset, ganando throughput a costa de ruido (estilo Hogwild [5]). En este trabajo se usa el régimen sincrónico (barrera con store de doble buffer), de modo que AR y PS aplican *la misma regla de actualización* (el mismo update de SGD).

II-C. All-Reduce

En *ring all-reduce* los W workers se ordenan en anillo y ejecutan dos fases: *scatter-reduce* (cada worker termina con un fragmento del gradiente sumado) y *all-gather* (se propaga el fragmento ya reducido) [3]. El volumen de datos de gradiente que comunica cada worker es $\approx 2 \frac{W-1}{W} |g|$, independiente de W salvo un factor constante, lo que proporciona buena eficiencia de ancho de banda. No hay servidor central ni punto único de contención, pero todos avanzan al ritmo del más lento (sincronía estricta). O.N.O. promedia el gradiente reducido dividiendo por W .

II-D. O.N.O.

Los experimentos se corren sobre O.N.O., un sistema distribuido propio que implementa tanto All-Reduce como Parameter Server bajo una misma interfaz. La Figura 1 contrasta las dos estrategias de agregación comparadas.

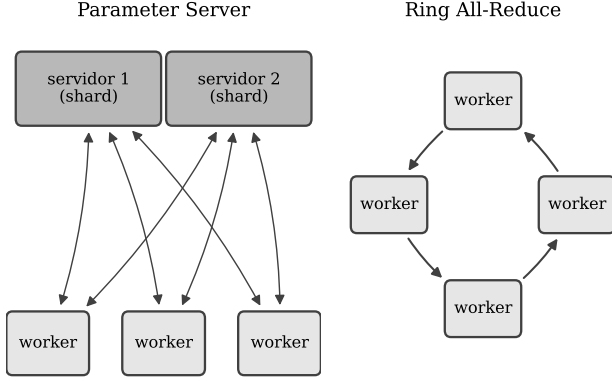


Figura 1: Parameter Server (izquierda): los workers hacen push/pull contra dos servidores que shardean los pesos. Ring All-Reduce (derecha): los workers promedian gradientes entre sí sin servidor central.

III. METODOLOGÍA EXPERIMENTAL

III-A. Datasets y modelos

Como benchmark principal se usa **FashionMNIST** [6], una tarea más exigente y menos separable que los dígitos, y **MNIST** [7] como referencia de correctitud y convergencia. La Tabla I resume los datasets y la Tabla II las arquitecturas. El modelo principal es *nielsen* (una CNN pequeña con softmax y entropía cruzada). Para aislar la presión de comunicación se definen redes densas de tamaño creciente (Densa S/M/L), cuyo objetivo no es la accuracy sino variar el tamaño del gradiente.

Tabla I: Conjuntos de datos utilizados en la evaluación.

Dataset	Train	Test	Forma	Clases
MNIST	60 000	10 000	$28 \times 28 \times 1$	10
FashionMNIST	60 000	10 000	$28 \times 28 \times 1$	10

Tabla II: Modelos empleados y su tamaño aproximado. Las redes Densa S/M/L son totalmente conectadas.

Modelo	Arquitectura	Parámetros
Nielsen	Conv20@5 $\rightarrow$ Pool $\rightarrow$ Dense100 $\rightarrow$ Dense10	289 630,00
Densa S	Dense128 $\rightarrow$ Dense10	101 770,00
Densa M	Dense512 $\rightarrow$ Dense512 $\rightarrow$ Dense10	669 706,00
Densa L	Dense1024 $\rightarrow$ Dense1024 $\rightarrow$ Dense10	1 863 690,00

III-B. Estrategias y configuraciones

Se comparan dos estrategias distribuidas: All-Reduce (todos los nodos son workers) y Parameter Server (barrera, store de doble buffer, con los pesos shardeados entre dos servidores).

Las topologías se detallan en la Tabla III. El criterio de equidad es *igual número de workers*: para cada N , All-Reduce usa N workers y Parameter Server usa los mismos N workers más dos servidores (los servidores son el costo estructural propio de PS, no un recorte de workers). Se evalúa $N \in \{3, 5\}$.

Tabla III: Topologías distribuidas bajo el diseño de *workers* iguales. PS emplea 2 servidores con parámetros fragmentados (*sharding*); los *workers* se igualan a All-Reduce para preservar la equidad. Batch efectivo = *workers* $\times$ 10.

Estrategia	Workers	Servidores	Nodos	Batch efectivo
AR	3	0	3	30
AR	5	0	5	50
PS	3	2	5	30
PS	5	2	7	50

III-C. Criterio de comparación justa

En O.N.O. el *batch_size* es *por worker*, de modo que el batch efectivo es $W \cdot b$. Comparar a igual número de workers tiene una consecuencia deseable: como All-Reduce y Parameter Server usan los mismos W workers con el mismo b , el batch efectivo $W \cdot b$ es idéntico para ambas estrategias en cada topología. La semántica de optimización queda entonces fijada, y por consiguiente la convergencia es directamente comparable. Se reportan dos mediciones complementarias: con un **batch por worker pequeño** se mide convergencia y accuracy; con un **batch por worker mayor** se mide throughput bruto. Se es explícito sobre qué pregunta responde cada una.

III-D. Métricas y entorno

Se mide la pérdida de entrenamiento por época, la accuracy de test (la fracción de aciertos, es decir el *argmax* de la salida, sobre el conjunto de test completo), el tiempo total, los samples por segundo, el speedup y la eficiencia de escalado. Entorno: una única máquina de 8 núcleos; el clúster se *simula* con contenedores Docker (un contenedor por nodo, red bridge, puertos publicados). Al aumentar el número de nodos los ocho núcleos físicos se saturan: se mide escalado *lógico* bajo recursos compartidos, no escalado multi-máquina (ver Sección V-A). Se utiliza optimizador SGD y semilla fija para reproducibilidad.

IV. RESULTADOS

IV-A. Corrección y convergencia

Ambas estrategias entrenan correctamente. Con la receta de referencia, a igual número de workers All-Reduce y Parameter Server alcanzan accuracies de test prácticamente iguales en ambos datasets (con Parameter Server incluso marginalmente por encima en FashionMNIST), según la Tabla IV. La Figura 2 muestra que las curvas de pérdida descienden de forma casi solapada. Este resultado es esperable: como el régimen es sincrónico, ambos esquemas de agregación aplican la misma regla de actualización, de modo que no cabe esperar una diferencia sistemática de convergencia entre ellos en este entorno.

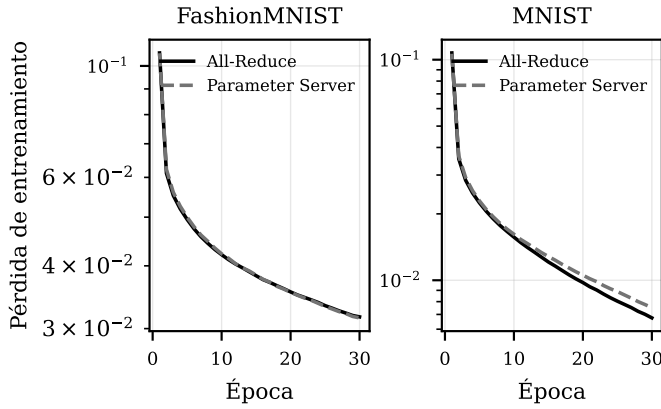


Figura 2: Convergencia (pérdida de entrenamiento por época) a 3 workers, en FashionMNIST (izquierda) y MNIST (derecha), para All-Reduce y Parameter Server.

IV-B. Comparación justa y tiempo hasta accuracy

A igual número de workers, All-Reduce y Parameter Server convergen a una accuracy prácticamente igual en ambos datasets (Tabla IV). La diferencia decisiva es el *tiempo*: All-Reduce alcanza esa accuracy apreciablemente antes que Parameter Server (Tabla IV), debido a que el servidor central de PS actúa como punto de serialización por el que pasan todos los updates.

Tabla IV: Resultados de convergencia justa (batch 10 por worker con evaluación en test) para ambas estrategias en cada conjunto de datos.

Dataset	Estrategia	Workers	Acc. test (%)	Tiempo total (s)
FashionMNIST	AR	3	87,28	210,9
FashionMNIST	PS	3	87,58	257,6
FashionMNIST	AR	5	86,25	248,1
FashionMNIST	PS	5	86,49	272,5
MNIST	AR	3	97,82	213,8
MNIST	PS	3	97,62	251,5
MNIST	AR	5	97,00	253,4
MNIST	PS	5	96,75	269,1

IV-C. Throughput y escalabilidad

En condiciones de batch por worker fijo, la Figura 3 muestra un cruce revelador. Con pocos workers All-Reduce sostiene mayor throughput que Parameter Server, pero al sumar workers las tendencias se invierten: el throughput de All-Reduce cae levemente, debido a que su anillo sincrónico satura los núcleos físicos, mientras que el de Parameter Server crece y llega a igualarlo. La Figura 4 lo expresa como speedup relativo a la configuración con menos workers: Parameter Server escala con pendiente positiva al sumar nodos, amortizando el costo fijo de sus servidores, aunque para ello emplea dos nodos servidores adicionales que All-Reduce no necesita.

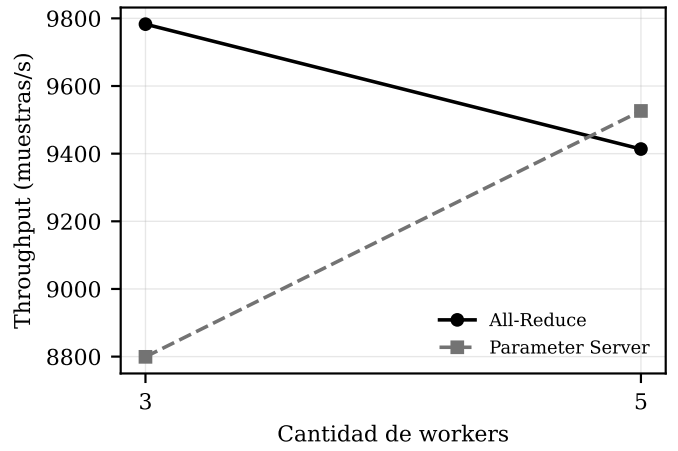


Figura 3: Throughput (samples/s) vs. número de workers, batch por worker fijo, en FashionMNIST. All-Reduce parte más alto pero Parameter Server lo alcanza al crecer los workers.

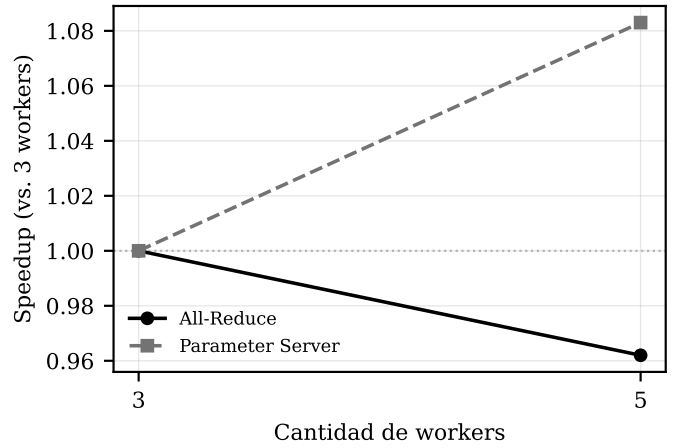


Figura 4: Speedup de throughput respecto de la configuración de 3 workers. Parameter Server escala con mayor pendiente que All-Reduce desde una base más baja.

IV-D. Presión de comunicación

Para aislar el efecto del tamaño del gradiente se entrenan redes densas de tamaño creciente (Tabla II) a 3 workers, con batch por worker fijo. La Figura 5 (ejes logarítmicos) muestra que el throughput se desploma al crecer el modelo: la comunicación, no el cómputo, domina. All-Reduce sostiene mayor throughput que Parameter Server en los tres tamaños, pero su ventaja se *reduce* al crecer el modelo (Figura 5). La razón es estructural: con modelos más grandes, el sharding de parámetros entre los dos servidores de PS reparte mejor la presión de comunicación y le permite acortar la brecha.

Este comportamiento se entiende mejor con una cuenta independiente de la red. La Figura 6 grafica el volumen de datos comunicado por paso: en All-Reduce, cada nodo comunica $\approx 2 \frac{W-1}{W} |g|$, esencialmente constante; en Parameter Server, el tráfico que ingresa a cada servidor crece de forma lineal con el número de workers (atenuado por el sharding entre S

servidores). Esta métrica analítica explica tanto la eficiencia de ancho de banda de All-Reduce como el porqué el servidor se vuelve el punto de presión al sumar workers, con independencia de la velocidad de la red subyacente. La compresión de gradientes, dispersa (*sparse*) o cuantizada, es una mitigación conocida para esta presión [8]: O.N.O. la implementa, pero su evaluación queda fuera del alcance de esta monografía.

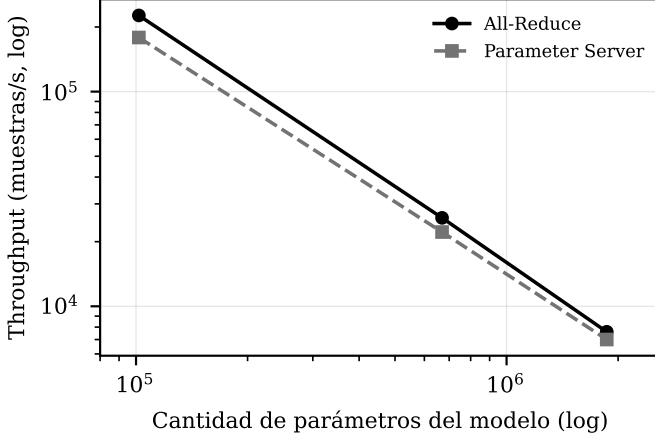


Figura 5: Presión de comunicación: throughput (log) vs. tamaño del modelo (log) a 3 workers. All-Reduce sostiene mayor throughput en los tres tamaños; la ventaja se reduce al crecer el modelo.

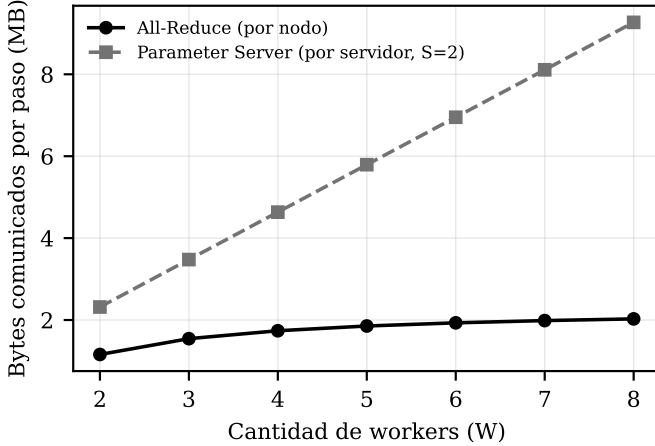


Figura 6: Volumen de datos comunicado por paso (métrica analítica, independiente de la red) vs. número de workers. En All-Reduce el volumen por nodo es casi constante; en Parameter Server el tráfico por servidor crece con los workers.

V. DISCUSIÓN

Cuándo conviene All-Reduce. La evidencia favorece a All-Reduce en el escenario evaluado: clúster homogéneo, red rápida (loopback), entrenamiento síncrono y gradientes densos. Iguala a Parameter Server en convergencia y entrega mayor throughput y menor tiempo hasta accuracy, sin un servidor central que congestione. Por eso, cuando los datos y

los nodos son homogéneos (y no hay, por tanto, heterogeneidad ni stragglers que un esquema asíncrono pueda aprovechar), All-Reduce podría considerarse la elección natural. Es, además, el caso en el que MNIST y FashionMNIST distribuidos de forma uniforme sitúan a este estudio.

Cuándo conviene Parameter Server. La evidencia favorece a Parameter Server en dos aspectos parciales del entorno evaluado: escala con mayor pendiente al sumar nodos, amortizando el costo fijo de sus servidores, y su sharding de parámetros ya empieza a acortar la brecha de throughput con los modelos más grandes. Su ventaja estructural más fuerte (el sharding de modelos que no entran en un solo nodo) sería decisiva a mayor escala, pero el modelo mayor evaluado sí completó en All-Reduce, por lo que no se ejercitó. El resto de sus ventajas (heterogeneidad, stragglers, asincronía) son aspectos a abordar en trabajos futuros.

Tabla V: Matriz de decisión cualitativa All-Reduce vs. Parameter Server (acotada al entorno evaluado).

Criterio	All-Reduce	Parameter Server
Convergencia	Empate: misma semántica síncrona.	Empate: misma semántica síncrona.
Throughput / tiempo hasta <i>accuracy</i>	Favorable: sin servidor central.	Menor: sobrecarga del servidor.
Escalado al sumar nodos	Escala bien, pendiente base.	Amortiza su costo fijo: pendiente algo mayor.
Eficiencia de hardware	No requiere nodos servidores extra.	Requiere nodos servidores dedicados.
Presión de comunicación / modelos grandes	Sostiene la ventaja.	El <i>sharding</i> acorta la brecha.
Heterogeneidad / stragglers / asincronía	Síncrono: sensible a <i>stragglers</i> .	Ventaja estructural (asincronía); no ejercitada aquí.

V-A. Limitaciones

El clúster se simula en una sola máquina de 8 núcleos: el escalado es lógico, no físico, y la red es loopback, sin latencia ni ancho de banda realistas. Esta es la limitación principal, y es la que más pesa sobre la comparación de rendimiento: una red rápida atenúa el costo de comunicación de Parameter Server, de modo que la métrica analítica de Figura 6 debe leerse como complemento de los tiempos medidos. El estudio se restringe además al régimen síncrono y a datos distribuidos de forma homogénea; no se analizan stragglers, heterogeneidad de nodos ni particiones no uniformes, que en este trabajo no fueron evaluados. O.N.O. es un sistema académico: no compite con frameworks industriales como Horovod [4] o BytePS [9], que se usan solo como marco conceptual.

VI. CONCLUSIÓN

Respondiendo a la pregunta de investigación, en el entorno evaluado (homogéneo, síncrono y sobre una sola máquina con red rápida) es posible concluir que All-Reduce es preferible: converge a la misma accuracy que Parameter Server (a igual número de workers y en ambos datasets) y lo supera en throughput y en tiempo hasta accuracy, debido a que el servidor central de PS introduce un cuello de serialización. Parameter Server

recorta distancia al escalar y con modelos grandes gracias al sharding, pero no superó a All-Reduce en throughput aún con el modelo más grande; podría considerarse una elección natural solo si se explotaran sus ventajas estructurales (heterogeneidad, asincronía, sharding de modelos que no entran en un nodo), que aquí no fueron ejercitadas. O.N.O. aporta evidencia de que un sistema distribuido propio permite contrastar ambas familias bajo un criterio de comparación justa. Se plantea como trabajo futuro evaluar redes con latencia realista, stragglers controlados, particiones de datos no homogéneas y modelos de mayor escala.

REFERENCIAS

- [1] T. Ben-Nun and T. Hoefer, “Demystifying parallel and distributed deep learning: An in-depth concurrency analysis,” *ACM Computing Surveys*, vol. 52, no. 4, pp. 1–43, 2019.
- [2] M. Li, D. G. Andersen, J. W. Park, A. J. Smola, A. Ahmed, V. Josifovski, J. Long, E. J. Shekita, and B.-Y. Su, “Scaling distributed machine learning with the parameter server,” in *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, 2014, pp. 583–598.
- [3] A. Gibiansky, “Bringing HPC techniques to deep learning,” Baidu Research, technical report, 2017, ring all-reduce for distributed gradient aggregation.
- [4] A. Sergeev and M. Del Balso, “Horovod: Fast and easy distributed deep learning in TensorFlow,” *arXiv preprint arXiv:1802.05799*, 2018.
- [5] F. Niu, B. Recht, C. Ré, and S. J. Wright, “Hogwild!: A lock-free approach to parallelizing stochastic gradient descent,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011, pp. 693–701.
- [6] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms,” 2017.
- [7] Y. LeCun, C. Cortes, and C. J. Burges, “The MNIST database of handwritten digits,” <http://yann.lecun.com/exdb/mnist/>, 1998.
- [8] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, “Deep gradient compression: Reducing the communication bandwidth for distributed training,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [9] Y. Jiang, Y. Zhu, C. Lan, B. Yi, Y. Cui, and C. Guo, “A unified architecture for accelerating distributed DNN training in heterogeneous GPU/CPU clusters,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 463–479.